

KickProphet

Un Sistema di Machine Learning per la
Predizione del Successo su Kickstarter

Autori:

Alessio Del Sorbo

Gianni Policola

Report Tecnico di Progetto

17/02/2026

Dipartimento di Informatica
Università degli Studi di Salerno

Indice

1	Introduzione e Scenario	4
1.1	Scenario e Definizione del Problema	4
1.2	Definizione del Task	4
2	Il Dataset	5
2.1	Origine e Caratteristiche	5
2.2	Descrizione delle Variabili	7
3	Metodologia	8
3.1	Pulizia e Preparazione dei Dati	8
3.1.1	Unificazione e Gestione dei Tipi	9
3.1.2	Deduplicazione Temporale	9
3.1.3	Filtro della Variabile Target	10
3.2	Feature Engineering e Preprocessing	10
3.2.1	Gestione delle Variabili Temporali e Controllo Qualità	10
3.2.2	Variabili Numeriche e Imputazione	11
3.2.3	Analisi e Trattamento del Testo (NLP)	12
3.2.4	Feature Temporali Aggiuntive	13
3.2.5	Encoding delle Variabili Categoricali	13
3.2.6	Split del Dataset e Prevenzione del Data Leakage	13
3.3	Selezione e Addestramento del Modello	13
3.3.1	Analisi delle Soluzioni Candidate	13
3.3.2	Scelta Finale: Ensemble Ibrido (LightGBM + XGBoost)	14
3.3.3	Ottimizzazione e Calibrazione	14
3.4	Pipeline di Inferenza e Controlli in Produzione	15
3.4.1	Gestione Valuta in Tempo Reale	16
3.4.2	Guardrails e Validazione dell'Input	16
3.4.3	Semantic Gating e Clamping	16
4	Valutazione e Conclusioni	16
4.1	Valutazione delle Prestazioni	16
4.1.1	Metriche di Classificazione	16
4.1.2	Affidabilità Probabilistica	18
4.2	Analisi del Modello e Interpretabilità	19
4.3	Analisi Economica e ROI	20
5	Considerazioni Finali e Sviluppi Futuri	21
5.1	Conclusioni del Progetto	21
5.2	Sviluppi Futuri	22

Elenco delle Figure

Figura 1	Distribuzione della variabile target (Success/Fail). Il dataset è bilanciato o sbilanciato?	5
Figura 2	Analisi delle Categorie: distribuzione dei progetti e tasso di successo per categoria principale.	6
Figura 3	Tasso di successo per Paese di origine. Esistono geografie più performanti?	6
Figura 4	Analisi della stagionalità: impatto del mese e del giorno di lancio sul tasso di successo.	6
Figura 5	Matrice di correlazione tra le variabili numeriche del dataset.	8
Figura 6	Distribuzione dell'obiettivo economico (Goal USD). La scala logaritmica evidenzia la skewness.	9
Figura 7	Analisi dei valori mancanti nel dataset grezzo prima del trattamento. . .	9
Figura 8	Analisi della durata delle campagne. Si notano outlier (durate eccessive) che vengono filtrati.	11
Figura 9	Rappresentazione dell'encoding ciclico temporale (Seno/Coseno) per mesi e giorni.	11
Figura 10	Effetto della trasformazione logaritmica sulle variabili asimmetriche (es. goal_usd).	12
Figura 11	Visualizzazione PCA degli embeddings semantici dei testi dei progetti.	12
Figura 12	Confronto delle prestazioni (Cross-Validation) tra i diversi modelli testati.	14
Figura 13	Curve di apprendimento (Learning Curve) di LightGBM: analisi di bias e varianza.	15
Figura 14	Confronto dell'importanza delle feature tra i diversi modelli dell'ensemble.	15
Figura 15	Matrice di Confusione sul Test Set. Mostra i Veri Positivi, Veri Negativi, Falsi Positivi e Falsi Negativi.	17
Figura 16	Curva ROC (Receiver Operating Characteristic) con $AUC = 0.895$	18
Figura 17	Curva di Calibrazione (Reliability Diagram). Una curva vicina alla bisettrice indica un modello ben calibrato.	19
Figura 18	Top Feature per importanza (Permutation Importance). Evidenzia i driver principali del successo.	20
Figura 19	Analisi del ROI: ritorno sull'investimento stimato in base all'utilizzo del modello predittivo.	21
Figura 20	Analisi della soglia di profitto ottimale per massimizzare il rendimento delle campagne selezionate.	21

1 Introduzione e Scenario

1.1 Scenario e Definizione del Problema

Il crowdfunding ha rivoluzionato il modo in cui i creators, gli imprenditori e gli artisti finanziano i loro progetti, democratizzando l'accesso al capitale e permettendo al pubblico di partecipare attivamente alla realizzazione di nuove idee. Tra le varie piattaforme, **Kickstarter** si è affermata come leader globale, ospitando milioni di campagne che spaziano dalla tecnologia ai giochi da tavolo, dal cinema all'editoria.

Nonostante la popolarità della piattaforma, il tasso di successo delle campagne è tutt'altro che garantito. Statisticamente, una porzione significativa dei progetti non raggiunge l'obiettivo di finanziamento prefissato, lasciando i creatori senza fondi e i sostenitori delusi. Le ragioni del fallimento sono molteplici: obiettivi economici non realistici, presentazioni poco curate, categorie sature, o semplicemente una mancata comprensione delle dinamiche della piattaforma.

In questo contesto, il problema fondamentale affrontato dal progetto *KickProphet* è la capacità di valutare la probabilità di successo di una campagna *prima* del suo lancio effettivo. L'obiettivo è fornire ai creatori uno strumento predittivo che analizzi i parametri iniziali del progetto (come titolo, descrizione, categoria, obiettivo economico e durata) e ne stimi le possibilità di riuscita. Questo permette di ottimizzare la campagna in fase di ideazione, massimizzando le chance di successo senza dover attendere i primi riscontri reali dal mercato.

1.2 Definizione del Task

Per rispondere al problema identificato, il progetto implementa un task di **Classificazione Binaria Supervisionata**. Il sistema è progettato per analizzare le caratteristiche statiche di un progetto (note come *features*) e predire l'esito finale della campagna: **Successo** (il progetto raggiunge o supera l'obiettivo di finanziamento) o **Fallimento** (il progetto non raggiunge l'obiettivo).

Sebbene il cuore del sistema sia un classificatore che distingue tra queste due classi, l'output finale presentato all'utente è concepito in termini di «**Fattibilità**» del progetto. Il modello, infatti, non si limita a fornire una risposta sì/no, ma calcola una probabilità di successo. Sulla base di questo valore probabilistico, il sistema restituisce all'utente un indicatore di fattibilità e un feedback utile. Questo approccio trasforma una semplice predizione binaria in uno strumento decisionale, permettendo all'utente di capire non solo *se* il progetto avrà successo, ma anche *quanto è probabile* che ciò avvenga dati i parametri attuali.

2 Il Dataset

2.1 Origine e Caratteristiche

Il dataset utilizzato per questo progetto è composto da una collezione di file in formato **CSV** (*Comma-Separated Values*), contenenti i dati grezzi relativi a migliaia di campagne Kickstarter. I dati sono distribuiti su più file (da `Kickstarter.csv` a `Kickstarter083.csv`), per un totale di decine di migliaia di record. Ogni riga rappresenta una singola campagna di crowdfunding e ogni colonna una specifica caratteristica del progetto.

Le informazioni contenute spaziano dai dettagli anagrafici del progetto (titolo, descrizione, autore) ai dati finanziari (obiettivo, fondi raccolti, valuta), fino allo stato finale della campagna e alle tempistiche di lancio e chiusura.

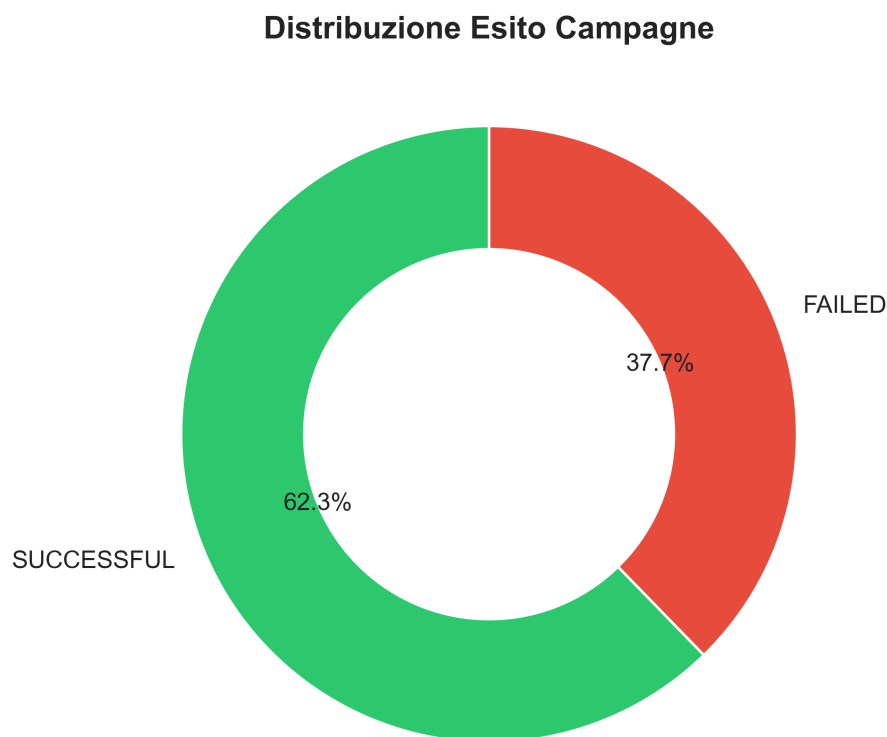


Figura 1: Distribuzione della variabile target (Success/Fail). Il dataset è bilanciato o sbilanciato?

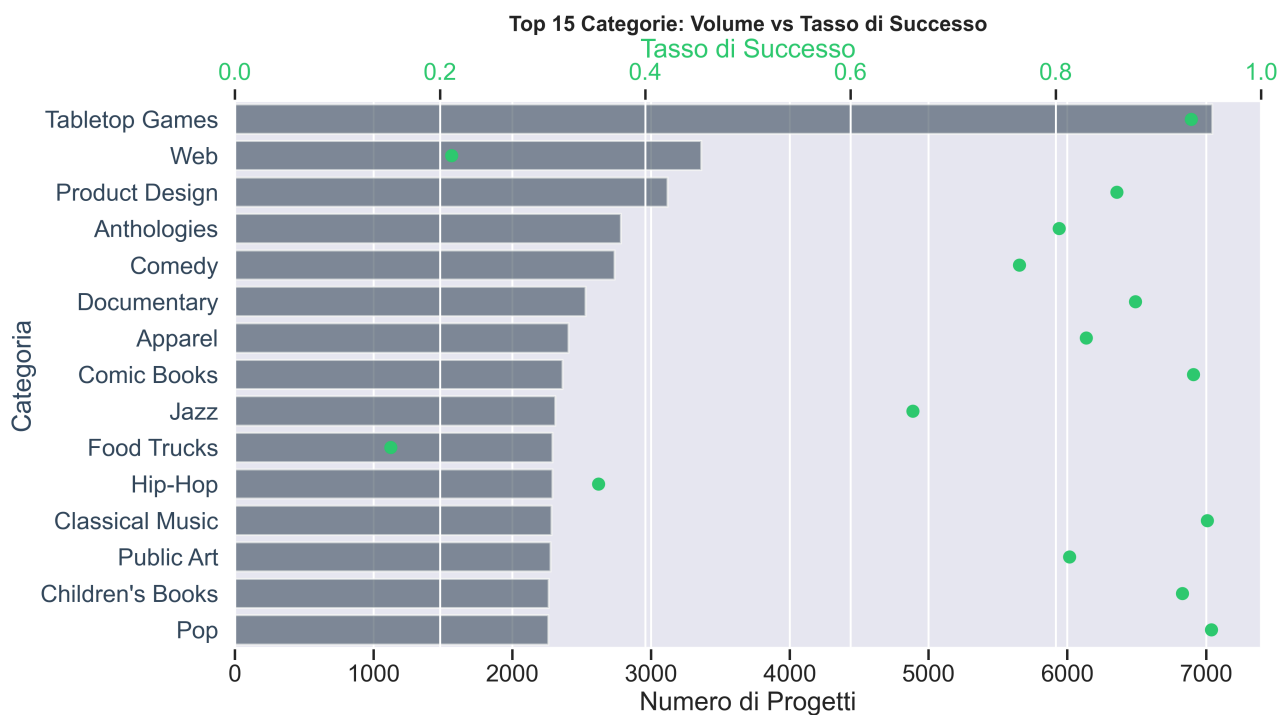


Figura 2: Analisi delle Categorie: distribuzione dei progetti e tasso di successo per categoria principale.

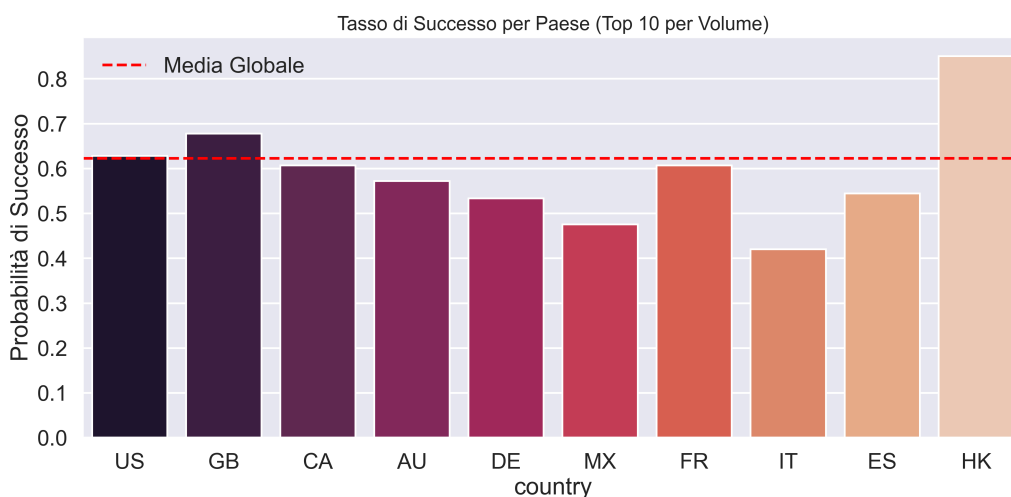


Figura 3: Tasso di successo per Paese di origine. Esistono geografie più performanti?

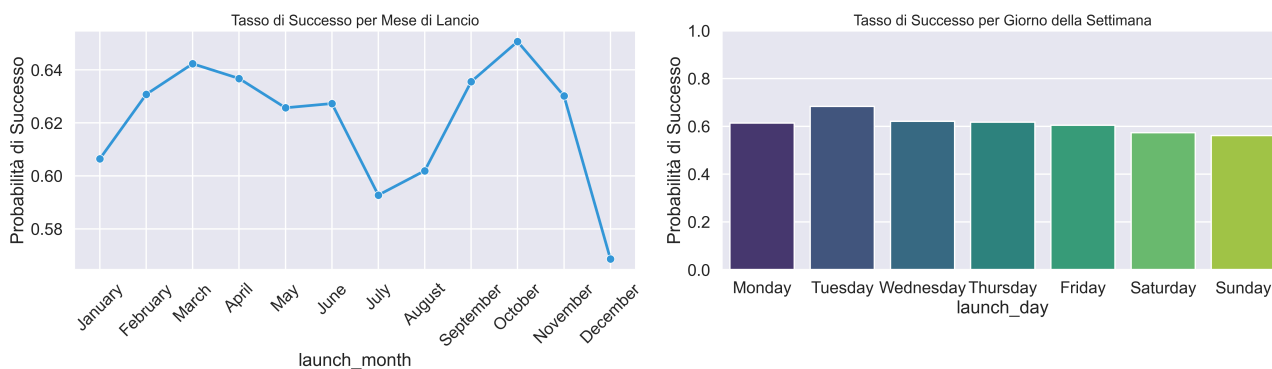


Figura 4: Analisi della stagionalità: impatto del mese e del giorno di lancio sul tasso di successo.

2.2 Descrizione delle Variabili

Il dataset grezzo presenta un totale di 45 colonne. Di seguito vengono descritte le variabili principali, suddivise per tipologia:

- **Identificativi e Testuali:**
 - **id:** Identificativo univoco del progetto.
 - **name:** Il titolo della campagna.
 - **blurb:** Una breve descrizione del progetto («blurb»).
 - **slug:** Una stringa identificativa URL-friendly del progetto.
- **Finanziarie:**
 - **goal:** L'obiettivo di finanziamento nella valuta originale del progetto.
 - **pledged:** L'importo totale raccolto nella valuta originale.
 - **usd_pledged:** L'importo raccolto convertito in dollari USA (USD).
 - **currency:** La valuta del progetto (es. USD, EUR, GBP).
 - **static_usd_rate:** Il tasso di cambio utilizzato per la conversione in USD.
 - **converted_pledged_amount:** Un'altra versione dell'importo raccolto convertito.
- **Stato e Esito:**
 - **state:** Lo stato attuale della campagna (es. *successful*, *failed*, *canceled*, *live*). Questa rappresenta la variabile target da predire.
 - **backers_count:** Il numero di sostenitori che hanno contribuito al progetto.
 - **percent_funded:** La percentuale di finanziamento raggiunto rispetto all'obiettivo.
- **Temporal:**
 - **created_at:** Timestamp di creazione del progetto sulla piattaforma.
 - **launched_at:** Timestamp del lancio ufficiale della campagna.
 - **deadline:** Timestamp della scadenza della campagna.
 - **state_changed_at:** Timestamp dell'ultimo cambio di stato.
- **Categoriche e Strutturate (JSON):**
 - **country:** Il paese di origine del progetto.
 - **category:** Una stringa in formato JSON contenente dettagli sulla categoria e sottocategoria del progetto (id, nome, slug, ecc.).
 - **creator:** Una stringa JSON con informazioni sul creatore del progetto.
 - **location:** Una stringa JSON con dettagli sulla localizzazione geografica.
 - **urls:** Link associati al progetto (pagina web, premi).
 - **photo:** Informazioni sulle immagini associate al progetto.
- **Altre:**
 - **spotlight:** Indicatore se il progetto è stato messo in evidenza da Kickstarter.
 - **staff_pick:** Indicatore se il progetto è una «scelta dello staff».
 - **disable_communication:** Flag che indica se le comunicazioni sono disabilitate.

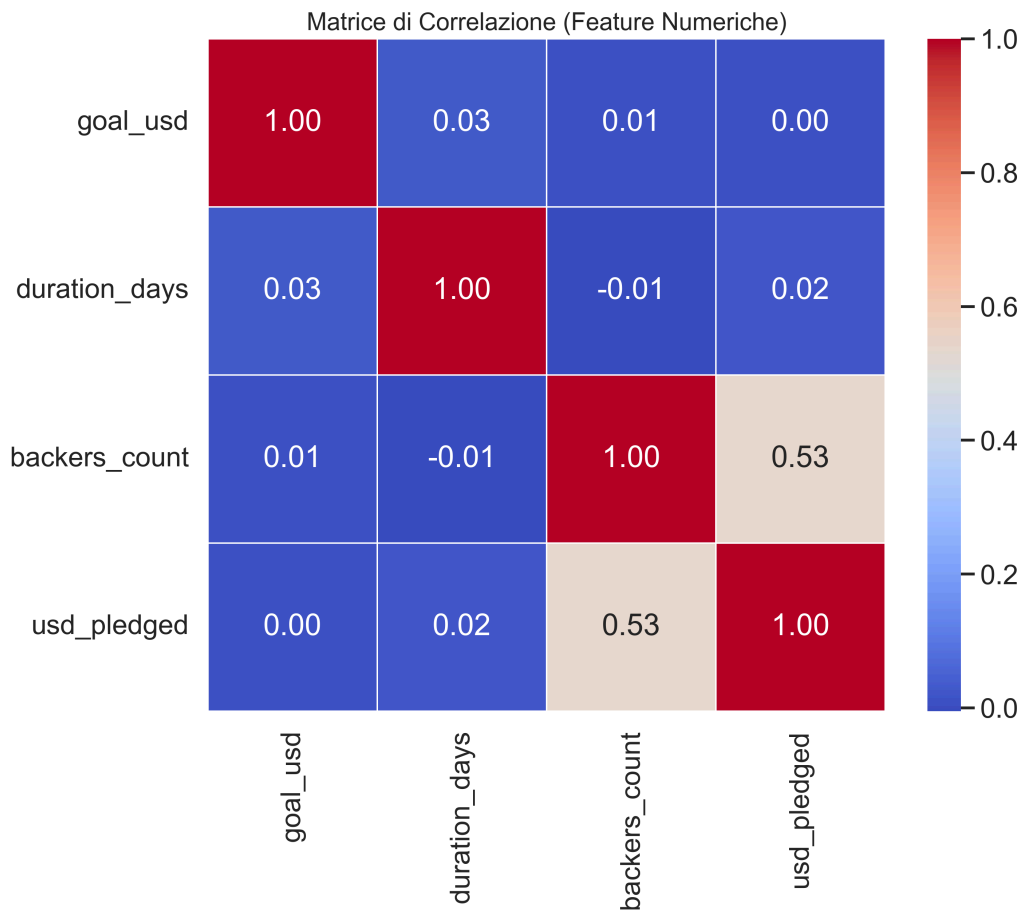


Figura 5: Matrice di correlazione tra le variabili numeriche del dataset.

3 Metodologia

3.1 Pulizia e Preparazione dei Dati

La prima fase critica della pipeline consiste nel trasformare i dati grezzi, frammentati e potenzialmente rumorosi, in un dataset coerente e affidabile. Questa operazione viene svolta in due step principali, implementati rispettivamente negli script `raw_dataset.py` e `clean_dataset.py`.

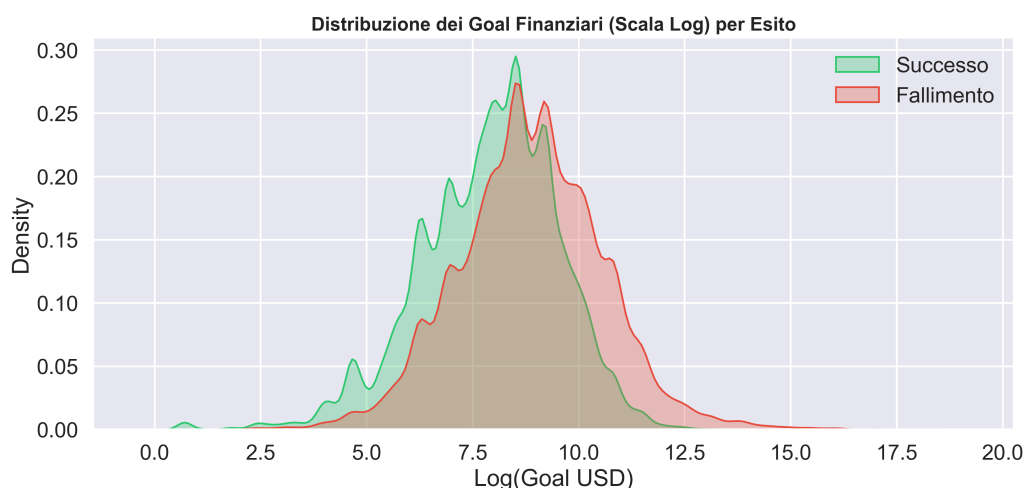


Figura 6: Distribuzione dell'obiettivo economico (Goal USD). La scala logaritmica evidenzia la skewness.

3.1.1 Unificazione e Gestione dei Tipi

Essendo i dati originali distribuiti su 84 file CSV distinti, il primo passo è stato l'unificazione in un unico dataframe (`raw_dataset.py`). Una criticità emersa in questa fase riguarda l'eterogeneità dei tipi di dati: Pandas, per default, tenta di inferire il tipo di ogni colonna (intero, float, stringa). Tuttavia, data la natura «sporca» di alcuni campi (es. ID che contengono caratteri, o date formattate diversamente), questa inferenza automatica rischia di generare errori di «Mixed Types» o corrompere informazioni. **Soluzione:** Tutti i dati vengono inizialmente letti forzatamente come stringhe (`dtype=str`). Questo preserva la fedeltà del dato originale al 100%, rimandando il casting dei tipi a una fase successiva dove può essere gestito in modo controllato.

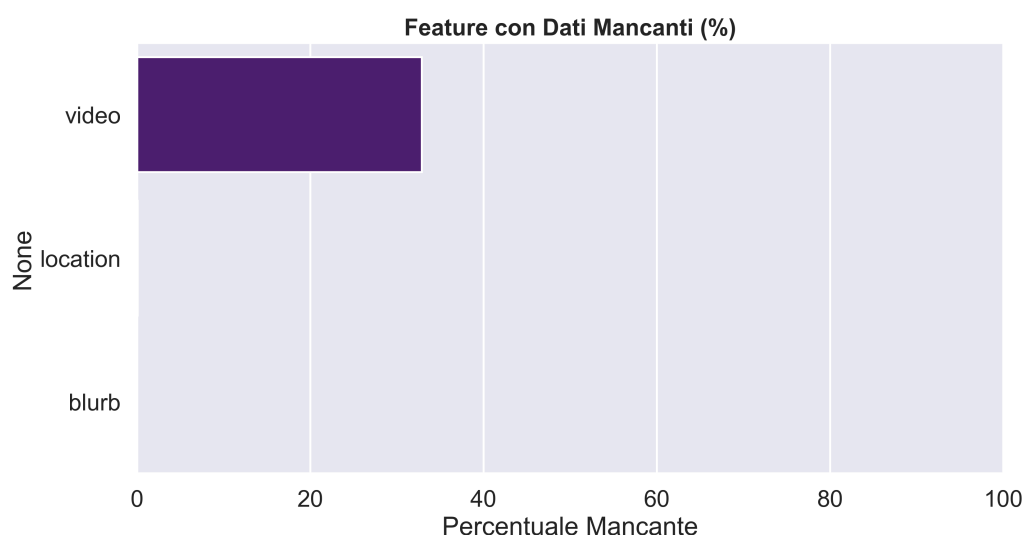


Figura 7: Analisi dei valori mancanti nel dataset grezzo prima del trattamento.

3.1.2 Deduplicazione Temporale

Il dataset grezzo contiene duplicati dello stesso progetto, dovuti al fatto che i dati sono stati raccolti tramite scraping in momenti diversi. È fondamentale mantenere

solo l'istanza più recente di ogni progetto per avere lo stato finale corretto. **Soluzione Implementata:**

1. Il dataset unificato viene ordinato per identificativo del progetto (`id`) e per una variabile temporale di riferimento (prioritariamente `state_changed_at`, altrimenti `deadline`).
2. Viene applicata una deduplicazione mantenendo esclusivamente l'ultimo record per ogni `id`. Questo garantisce che, se un progetto appare più volte (es. prima come «live» e poi come «successful»), venga conservato solo lo stato definitivo.

3.1.3 Filtro della Variabile Target

Poiché l'obiettivo è una classificazione binaria, è necessario rimuovere ambiguità nella variabile target `state`. **Soluzione:** Vengono filtrati e mantenuti solo i progetti con stato «*successful*» o «*failed*». Progetti con stati intermedi o non definitivi come «*live*», «*canceled*» o «*suspended*» vengono scartati, in quanto non forniscono un'etichetta ground truth affidabile per l'addestramento su campagne concluse.

3.2 Feature Engineering e Preprocessing

Una volta ottenuto un dataset pulito, la fase successiva (`preprocessing.py`) si concentra sull'arricchimento dei dati e sulla loro trasformazione in un formato numerico adatto agli algoritmi di Machine Learning. Questa fase è stata progettata per estrarre il massimo valore informativo dalle variabili grezze, con particolare attenzione ai dati testuali e categorici.

3.2.1 Gestione delle Variabili Temporal e Controllo Qualità

Le date grezze (`launched_at`, `deadline`, `created_at`) sono state elaborate per correggere incoerenze e derivare feature:

- **Filtri di Qualità:** Sono stati rimossi progetti con dati anomali, specificamente: durata > 60 giorni (non standard), obiettivo economico $< 10\$$ (non credibile) o date inconsistenti (es. lancio antecedente alla creazione del progetto).
- **Durata della Campagna:** Calcolata come differenza tra `deadline` e data di lancio.
- **Tempo di Preparazione:** Differenza tra la creazione della bozza e il lancio effettivo.
- **Encoding Ciclico:** Per catturare la stagionalità (mese, giorno della settimana) senza introdurre relazioni ordinali errate (es. Dicembre $>$ Gennaio), le variabili temporali sono state trasformate in coppie di coordinate (seno, coseno).

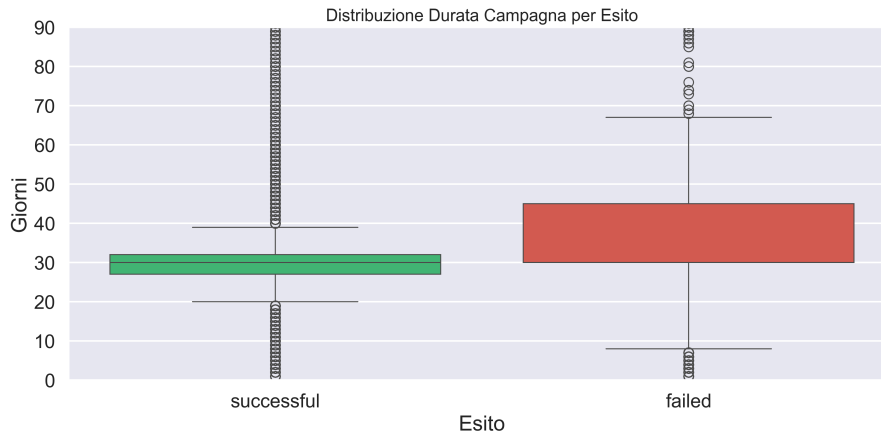


Figura 8: Analisi della durata delle campagne. Si notano outlier (durate eccessive) che vengono filtrati.

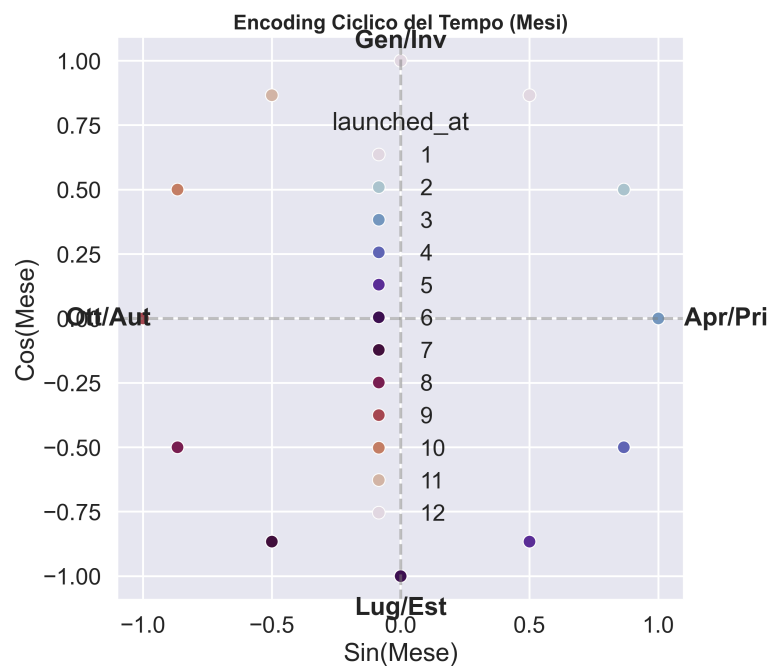


Figura 9: Rappresentazione dell'encoding ciclico temporale (Seno/Coseno) per mesi e giorni.

3.2.2 Variabili Numeriche e Imputazione

Per le variabili numeriche, sono state applicate strategie mirate per migliorare la qualità del segnale:

- **Imputazione:** I valori mancanti sono stati riempiti con la **Mediana** (robusta agli outlier).
- **Trasformazioni Logaritmiche:** Applicate a `goal_usd` e `preparation_days` per normalizzare distribuzioni asimmetriche.
- **Feature Derivate:** Sono state create variabili di interazione come `goal_per_day` (ambiziosità giornaliera del target) e indicatori binari come `has_video` e `prelaunch_activated` (se la pagina di pre-lancio era attiva).

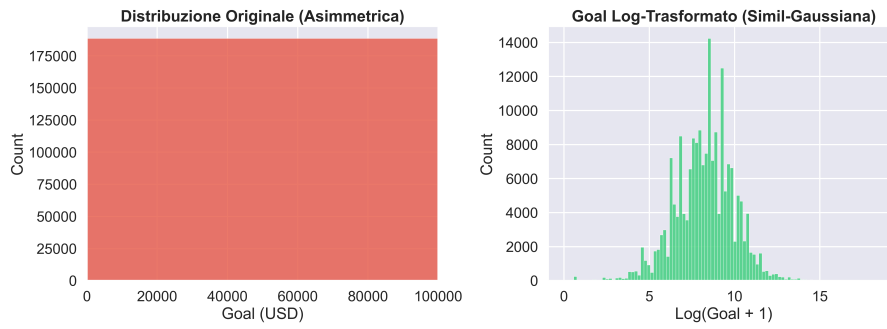


Figura 10: Effetto della trasformazione logaritmica sulle variabili asimmetriche (es. `goal_usd`).

3.2.3 Analisi e Trattamento del Testo (NLP)

Il processing del testo (titolo e descrizione) combina approcci classici e moderni:

1. **Metriche Statistiche e Leggibilità:** Calcolo di lunghezza, conteggio parole, rapporto cifre/lettere (*digit_ratio*) e indice di leggibilità (*Flesch Reading Ease*).
2. **TF-IDF (Term Frequency-Inverse Document Frequency):** Un approccio «bag-of-words» per identificare keyword specifiche e rilevanti nel corpus dei progetti.
3. **Embeddings Semantici (Transformer):** Utilizzo di `all-MiniLM-L6-v2` per generare vettori densi che catturano il contesto semantico.
4. **Riduzione Dimensionale (PCA):** Applicata agli embeddings per ridurre il rumore e la dimensionalità (da 384 a 64 componenti).
5. **Coerenza Semantica:** Calcolo della *Cosine Similarity* tra il testo del progetto e la sua categoria, per misurare la pertinenza.

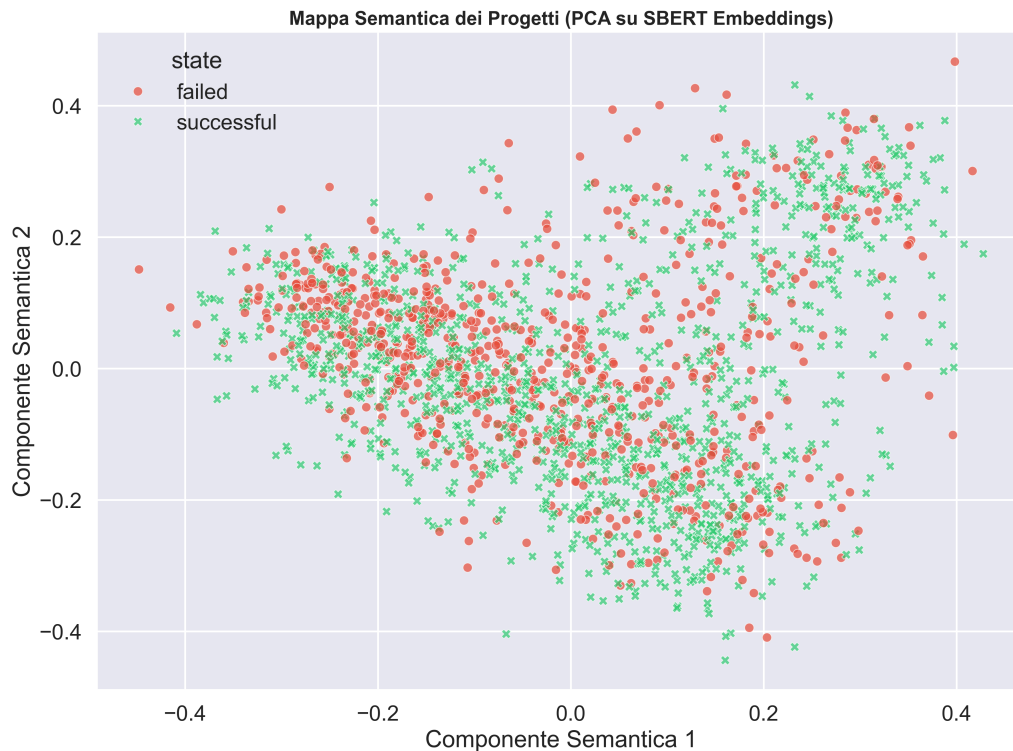


Figura 11: Visualizzazione PCA degli embeddings semantici dei testi dei progetti.

3.2.4 Feature Temporali Aggiuntive

Oltre all'encoding ciclico per mese e giorno, sono state estratte feature puntuali come un flag per il lancio nel weekend (`is_weekend`), ipotizzando un impatto sul comportamento dei sostenitori.

3.2.5 Encoding delle Variabili Categoriche

Il dataset presenta variabili categoriche ad alta cardinalità (es. `sub_category` con centinaia di valori unici) che rischiano di aumentare eccessivamente la dimensionalità del modello con il One-Hot Encoding classico. **Soluzione:**

- **Target Encoding:** Per la sottocategoria, è stato utilizzato il Target Encoding con smoothing. Ogni categoria viene sostituita dalla media della variabile target (probabilità di successo) per quella categoria, calcolata solo sul training set per evitare *data leakage*.
- **One-Hot Encoding:** Per variabili a bassa cardinalità (es. `main_category`, `country`), è stato mantenuto il classico One-Hot Encoding.

3.2.6 Split del Dataset e Prevenzione del Data Leakage

Per garantire una valutazione onesta delle prestazioni, il dataset è stato diviso in due sottoinsiemi: **Training Set (80%)** e **Test Set (20%)**. Fondamentalmente, tutte le trasformazioni che richiedono «conoscenza» della distribuzione dei dati (es. calcolo della media per il Target Encoding, fit della PCA o del TF-IDF) sono state «fittate» rigorosamente **solo sul Training Set** e poi applicate al Test Set. Questo previene il fenomeno del *data leakage*, assicurando che il modello non «veda» mai informazioni relative ai dati di test durante l'addestramento.

3.3 Selezione e Addestramento del Modello

La scelta del modello predittivo è stata frutto di un'analisi comparativa approfondita, volta a identificare l'algoritmo più adatto a gestire la complessità e l'eterogeneità dei dati di Kickstarter.

3.3.1 Analisi delle Soluzioni Candidate

In fase preliminare, sono state valutate diverse famiglie di algoritmi, ognuna con specifici pro e contro:

1. **Modelli Lineari (Logistic Regression):** Sebbene offrano un'eccellente interpretabilità e rapidità di addestramento, si sono rivelati inadeguati nel catturare le relazioni non lineari complesse presenti nei dati (es. l'interazione tra categoria e obiettivo economico, o la non-linearità delle feature testuali ridotte con PCA).
2. **Reti Neurali (Deep Learning):** Le reti neurali dense (MLP) sono state prese in considerazione per la loro capacità di apprendimento universale. Tuttavia, su dati tabulari di dimensioni medio-piccole, tendono a richiedere un tuning eccessivamente oneroso e spesso non superano le prestazioni dei metodi basati su alberi, risultando inoltre «black box» difficili da interpretare per l'utente finale.

3. **Ensemble Tree-Based (Random Forest, Gradient Boosting)**: Questa famiglia di algoritmi rappresenta lo stato dell'arte per i dati strutturati.

- *Random Forest*: Ottimo per ridurre la varianza e resistente all'overfitting, ma meno capace di ridurre il bias rispetto al boosting.
- *Gradient Boosting (XGBoost, LightGBM, CatBoost)*: Questi modelli costruiscono alberi in sequenza, correggendo iterativamente gli errori dei precedenti. Hanno dimostrato empiricamente le prestazioni migliori in termini di accuratezza e gestione di valori mancanti.

3.3.2 Scelta Finale: Ensemble Ibrido (LightGBM + XGBoost)

Sulla base dell'analisi, la scelta è ricaduta su una soluzione ibrida che combina due implementazioni avanzate di Gradient Boosting: **LightGBM** e **XGBoost**.

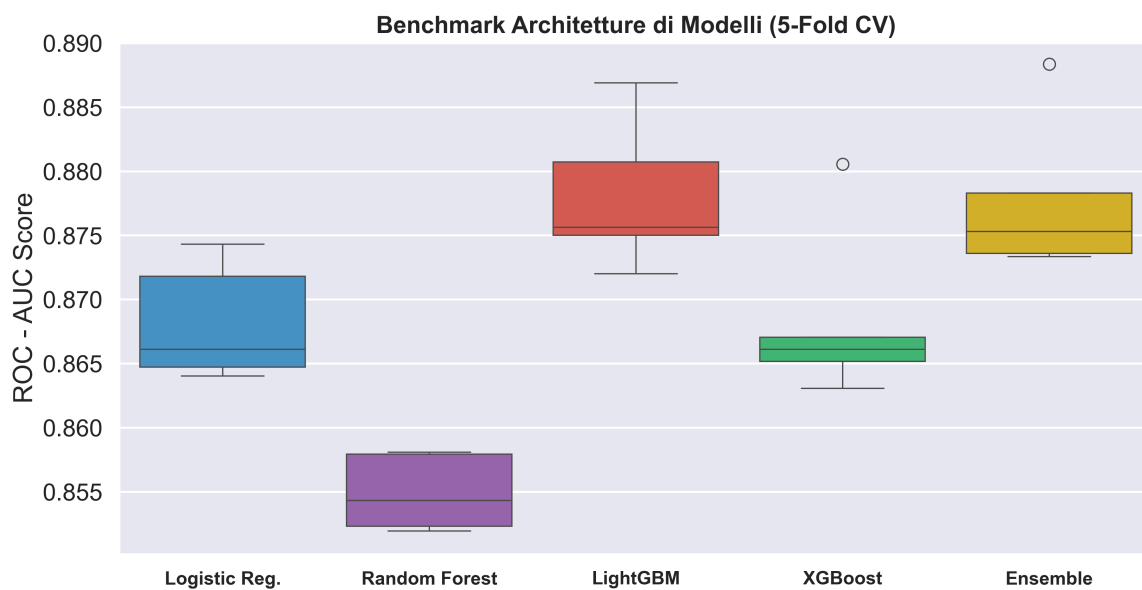


Figura 12: Confronto delle prestazioni (Cross-Validation) tra i diversi modelli testati.

La motivazione è duplice:

- **Robustezza**: LightGBM è estremamente veloce e gestisce ottimamente grandi quantità di dati, mentre XGBoost offre un sistema di regolarizzazione più conservativo. Combinarli permette di mitigare i difetti individuali e stabilizzare le predizioni.
- **Gestione Dati Eterogenei**: Entrambi gli algoritmi gestiscono nativamente i valori mancanti (senza necessitare di imputazione perfetta) e sono insensibili alla scala delle feature numeriche, semplificando la pipeline.

3.3.3 Ottimizzazione e Calibrazione

Una volta selezionata l'architettura, il processo di training si è articolato in tre step chiave:

1. **Tuning Bayesiano (Optuna)**: Per massimizzare le prestazioni, gli iperparametri di entrambi i modelli (profondità alberi, learning rate, regolarizzazione L1/L2) sono stati ottimizzati massimizzando la *Negative Log Loss* in Cross-Validation.

2. **Soft Voting:** I due modelli ottimizzati sono stati uniti in un Voting Classifier, dove la probabilità finale è la media ponderata delle probabilità dei singoli modelli.
3. **Calibrazione Isotonica:** Poiché l'obiettivo è fornire un indice di «Fattibilità» affidabile, è stato applicato uno strato di calibrazione (*Isotonic Regression*) per garantire che le probabilità predette rispecchino le frequenze reali di successo (es. tra i progetti a cui è assegnato il 70% di probabilità, effettivamente il 70% deve avere successo).

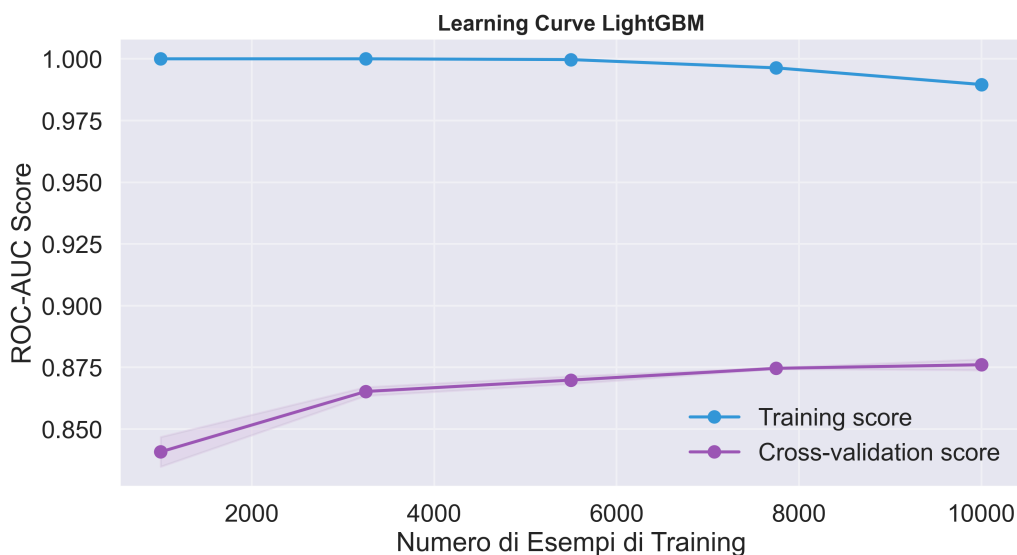


Figura 13: Curve di apprendimento (Learning Curve) di LightGBM: analisi di bias e varianza.

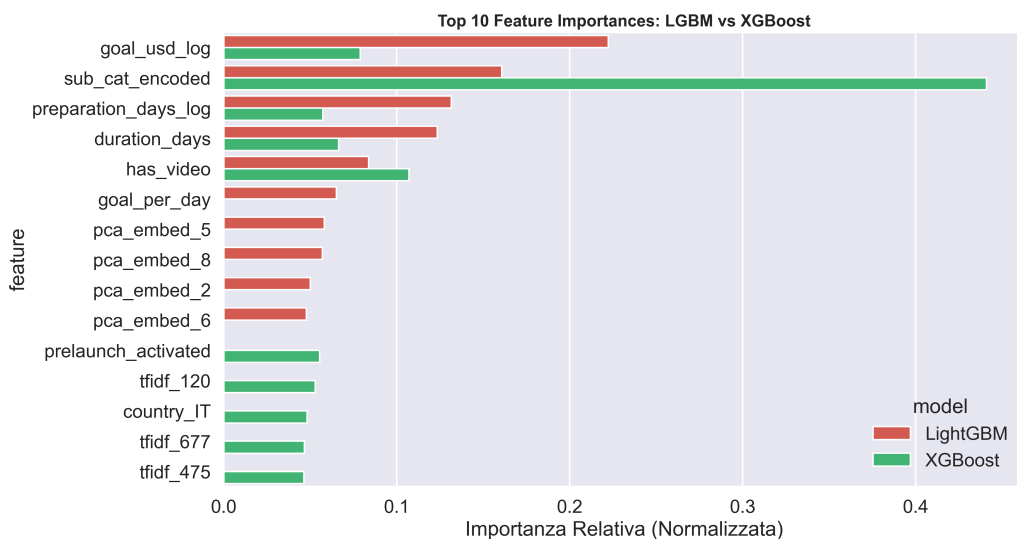


Figura 14: Confronto dell'importanza delle feature tra i diversi modelli dell'ensemble.

3.4 Pipeline di Inferenza e Controlli in Produzione

Oltre all'addestramento, il sistema include una pipeline di inferenza dedicata (`predict.py`) progettata per l'uso in produzione (es. nell'applicazione web). Questa pipeline implementa logiche aggiuntive non presenti nel training per garantire robustezza e sicurezza.

3.4.1 Gestione Valuta in Tempo Reale

A differenza del training (storico), in fase di predizione è critico valutare l'obiettivo economico con tassi di cambio aggiornati. Il sistema interroga un'API esterna (`api.exchangerate-api.com`) per convertire l'obiettivo in USD al tasso corrente. In caso di fallimento dell'API, è previsto un meccanismo di *fallback* su tassi statici pre-configurati.

3.4.2 Guardrails e Validazione dell'Input

Per proteggere il sistema da input di bassa qualità («garbage in, garbage out»), sono state implementate regole euristiche di blocco («Guardrails») che scartano a priori progetti non validi prima ancora di interrogare il modello:

- **Gibberish Detection:** Se la lunghezza media delle parole supera i 25 caratteri (indice di testo casuale o «spam»), la predizione viene abortita.
- **Controllo Ripetizioni:** Se il rapporto tra parole uniche e totali è inferiore al 15%, il testo è considerato ripetitivo/spam e rifiutato.
- **Incoerenza Semantica Estrema:** Se la *Cosine Similarity* tra descrizione e categoria è < 0.05 , il progetto è considerato totalmente fuori tema.

3.4.3 Semantic Gating e Clamping

È stata introdotta una logica di «Semantic Gating» per raffinare le feature: se la coerenza semantica del testo è bassa (< 0.17), il valore della feature `preparation_days` viene forzatamente azzerato. Questo impedisce che progetti con descrizioni lunghe ma irrilevanti beneficino ingiustamente del «bonus» derivante da un lungo periodo di preparazione. Inoltre, è stato applicato un **Clamping** (troncamento) ai giorni di preparazione: valori superiori a 60 giorni vengono appiattiti a 60, per evitare che anomalie temporali o progetti «dimenticati» in bozza influenzino eccessivamente la predizione.

4 Valutazione e Conclusioni

In questa sezione finale vengono presentati i risultati sperimentali ottenuti dall'Ensemble Ibrido (LightGBM + XGBoost) sul Test Set, che costituisce il 20% dei dati mai visti durante l'addestramento. L'analisi copre le metriche di classificazione standard, l'affidabilità delle probabilità predette e l'interpretabilità del modello.

4.1 Valutazione delle Prestazioni

4.1.1 Metriche di Classificazione

Il modello ha raggiunto prestazioni robuste, bilanciando efficacemente precisione e capacità di recupero (recall) in un contesto moderatamente sbilanciato. Di seguito il riepilogo delle metriche principali:

Metrica	Valore
Accuracy	81.4%
Precision	86.1%
Recall	83.7%
F1-Score	84.9%
ROC-AUC	89.5%
Brier Score	0.126

Tabella 1: Metriche aggregate sul Test Set.

L'Accuratezza dell'81.4% conferma che il sistema è in grado di prevedere correttamente l'esito di 4 campagne su 5. Particolarmente rilevante è l'AUC-ROC vicina a 0.90, che indica un'eccellente capacità di discriminazione tra classi positive (Successo) e negative (Fallimento) a vari livelli di soglia.

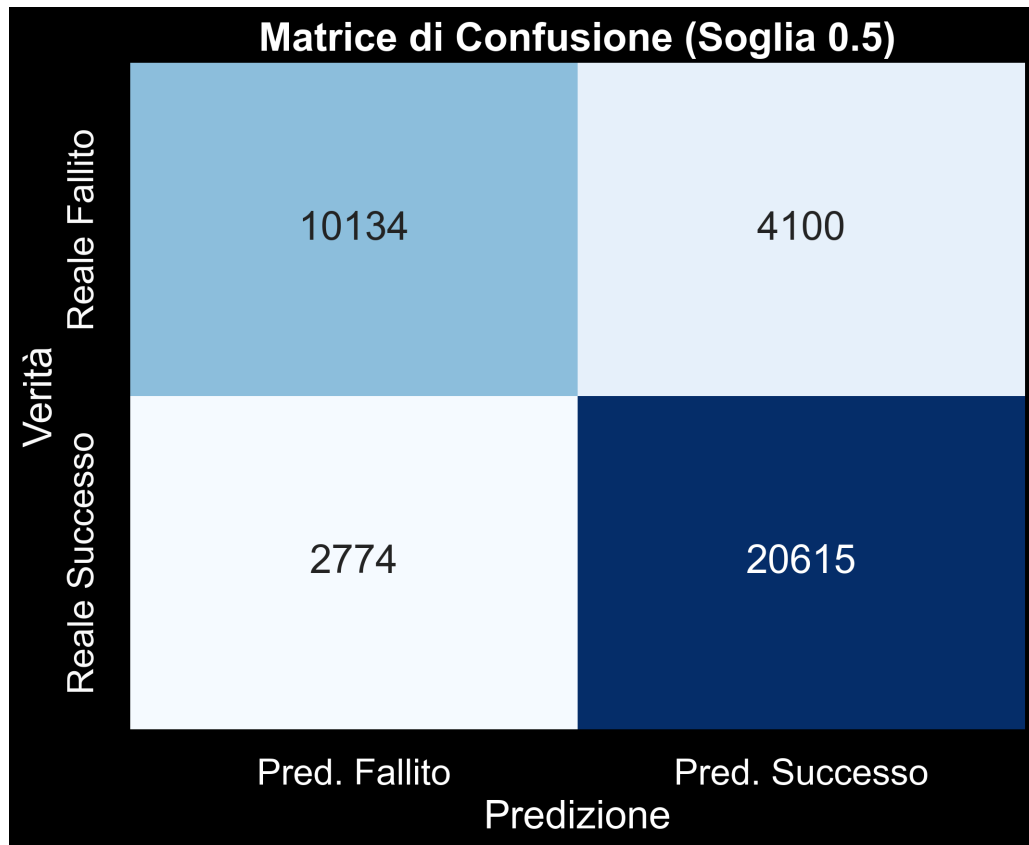


Figura 15: Matrice di Confusione sul Test Set. Mostra i Veri Positivi, Veri Negativi, Falsi Positivi e Falsi Negativi.

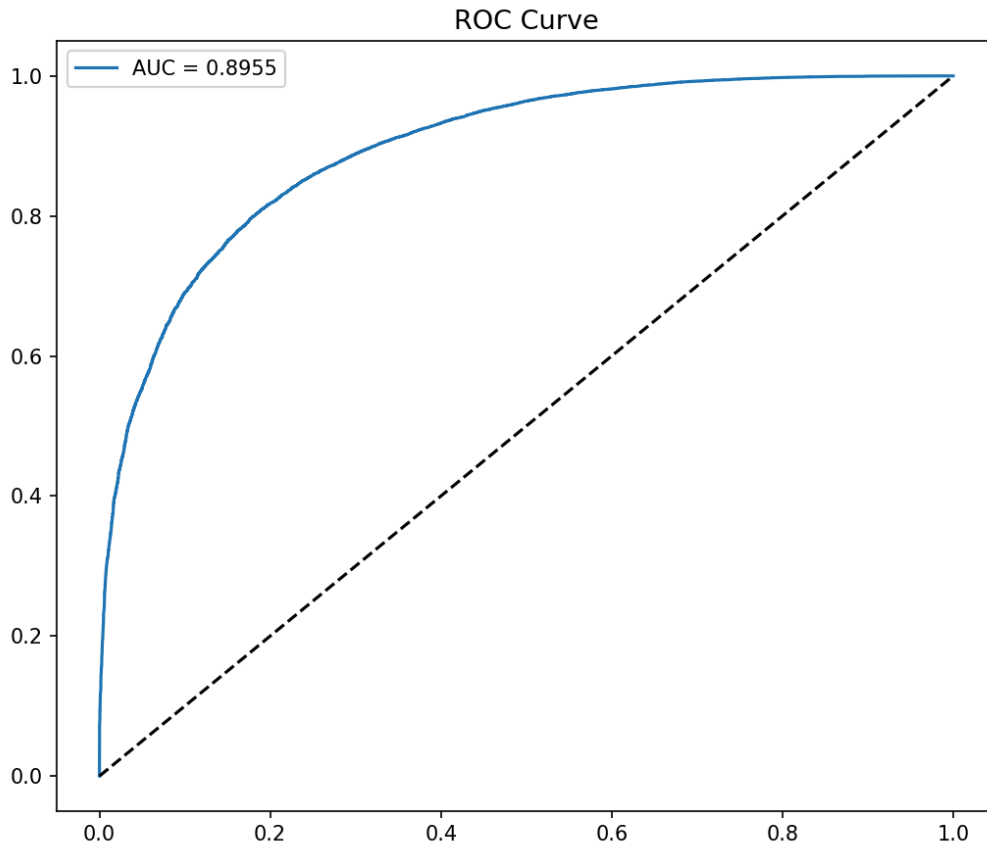


Figura 16: Curva ROC (Receiver Operating Characteristic) con $AUC = 0.895$.

4.1.2 Affidabilità Probabilistica

Essendo l'obiettivo del sistema fornire un indice di «fattibilità», la calibrazione delle probabilità è cruciale. Un Brier Score basso (0.126) e la curva di calibrazione (Figura 17) dimostrano che le probabilità stimate sono realistiche: una predizione di successo al 70% corrisponde effettivamente a un tasso di successo storico del 70%.

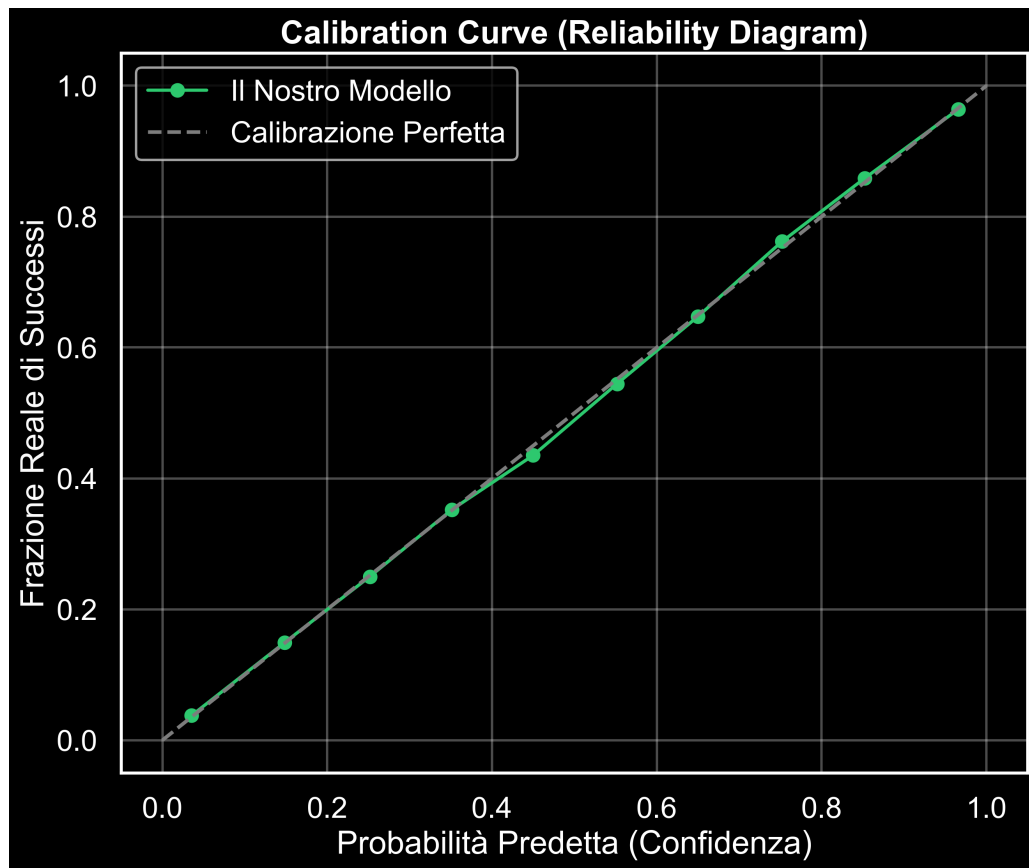


Figura 17: Curva di Calibrazione (Reliability Diagram). Una curva vicina alla bisettrice indica un modello ben calibrato.

4.2 Analisi del Modello e Interpretabilità

Per comprendere quali fattori guidano le predizioni, è stata analizzata l'importanza delle feature.

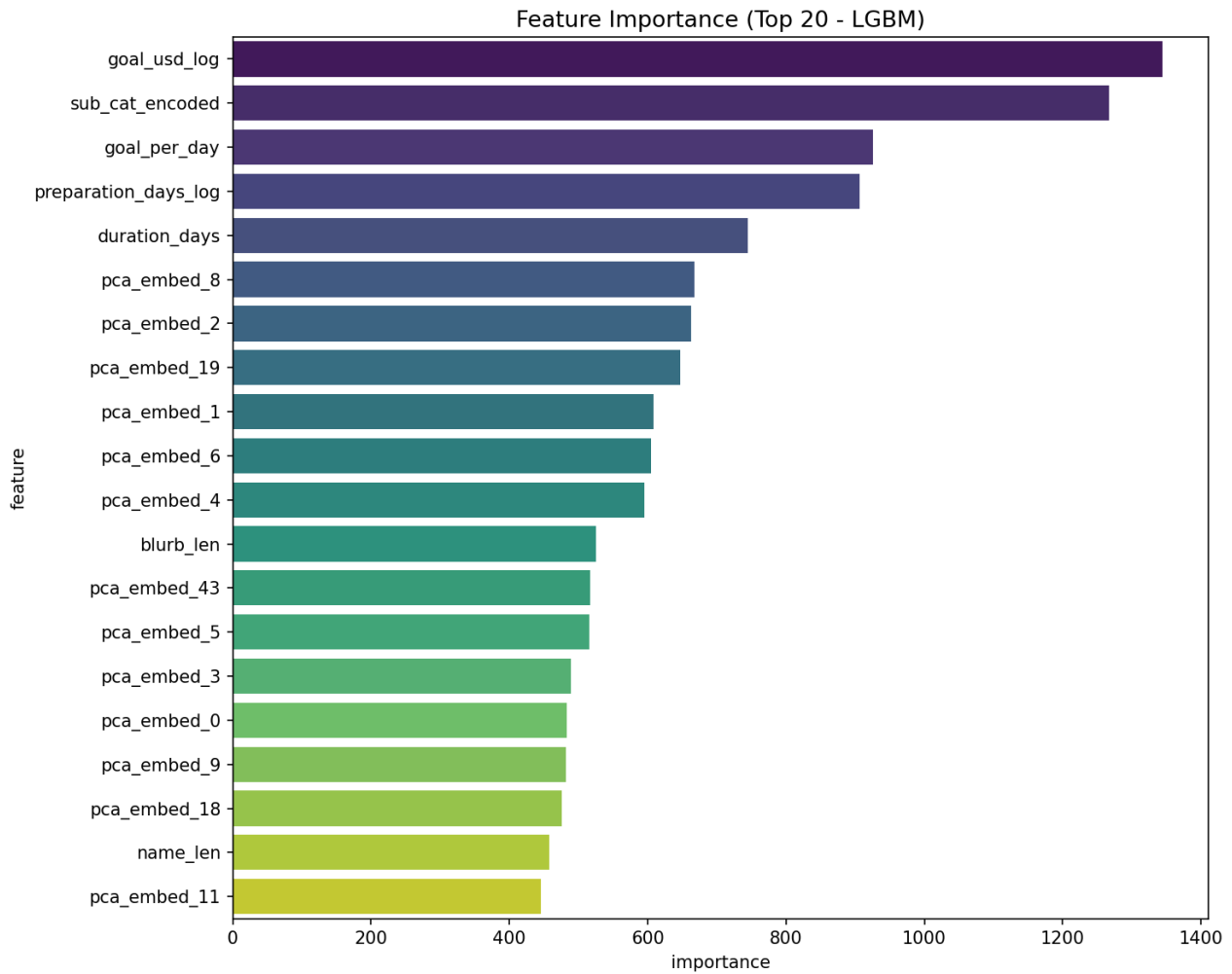


Figura 18: Top Feature per importanza (Permutation Importance). Evidenzia i driver principali del successo.

Dall'analisi emerge che il fattore predittivo dominante non è solo l'obiettivo economico (`goal_usd`), come prevedibile, ma anche variabili «comportamentali» e di qualità come la lunghezza della descrizione, la presenza di video e la tempestività del lancio (`preparation_days`). Le feature NLP e l'embedding semantico giocano un ruolo significativo nel «raffinare» la predizione, catturando la qualità della presentazione.

4.3 Analisi Economica e ROI

Oltre alle metriche tecniche, è fondamentale valutare l'impatto economico del modello. L'analisi del ROI (Return on Investment) e delle soglie di profitto aiuta a capire come l'adozione del modello possa ottimizzare le risorse.

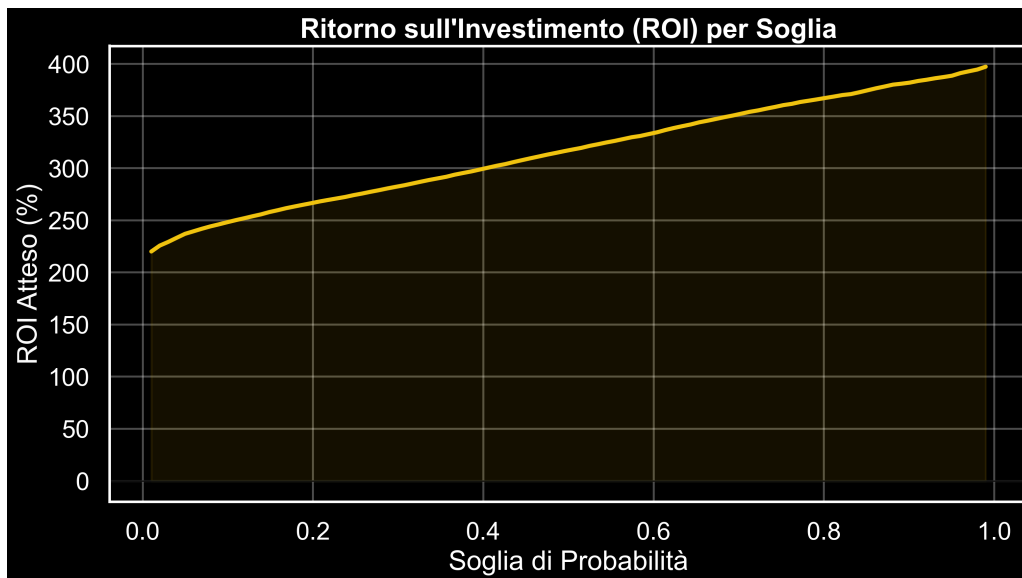


Figura 19: Analisi del ROI: ritorno sull'investimento stimato in base all'utilizzo del modello predittivo.

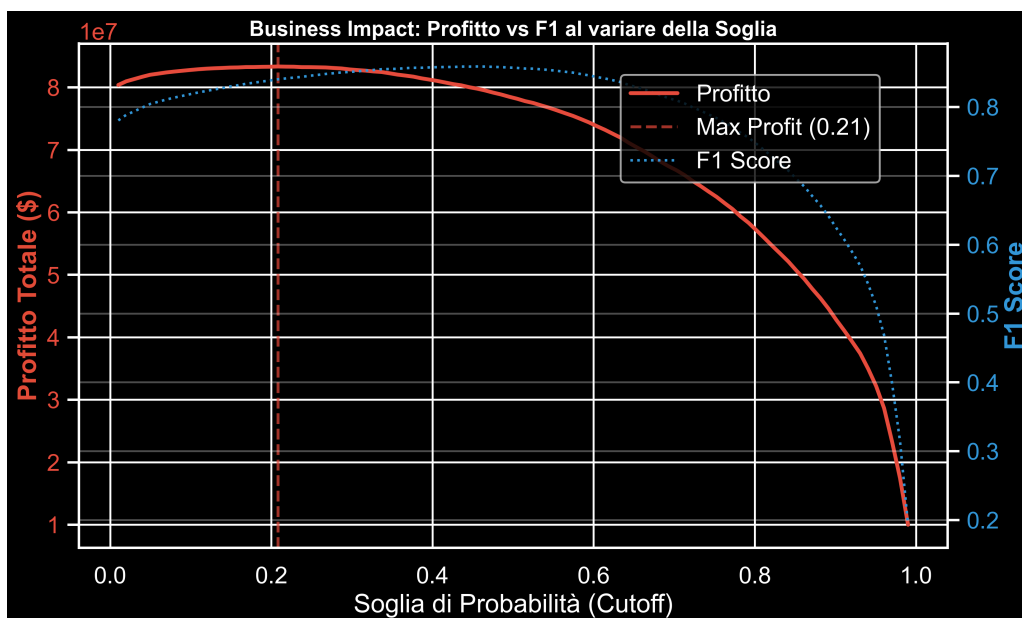


Figura 20: Analisi della soglia di profitto ottimale per massimizzare il rendimento delle campagne selezionate.

5 Considerazioni Finali e Sviluppi Futuri

5.1 Conclusioni del Progetto

Il progetto **KickProphet** ha raggiunto il suo obiettivo primario: sviluppare un sistema di Machine Learning in grado di prevedere il successo di una campagna Kickstarter utilizzando esclusivamente dati disponibili **prima del lancio**. Con un'accuratezza superiore all'**80%** e un'AUC-ROC prossima a **0.90**, il modello dimostra una robusta capacità di discriminazione. L'approccio ibrido (Ensemble di LightGBM e XGBoost), combinato con un'attenta ingegnerizzazione delle feature (NLP avanzato, encoding ciclico temporale) e una rigorosa calibrazione delle probabilità, ha permesso di

superare i limiti dei modelli base, offrendo predizioni affidabili anche in presenza di sbilanciamento delle classi.

Più che un semplice oracolo binario, il sistema si configura come uno strumento di supporto decisionale: il «punteggio di fattibilità» permette ai creatori di iterare sulla propria idea, raffinando descrizione, titolo e parametri economici per massimizzare le probabilità di successo prima di esporsi al mercato.

5.2 Sviluppi Futuri

Nonostante i risultati positivi, il sistema presenta margini di miglioramento ed evoluzione:

1. **Analisi Multimodale (Computer Vision):** Attualmente il modello ignora il contenuto visivo (immagini, thumbnail dei video). L'integrazione di una rete neurale convoluzionale (CNN) o di un Vision Transformer per estrarre feature qualitative dalle immagini del progetto potrebbe catturare segnali estetici cruciali per l'attrazione dei backer.
2. **Supporto Generativo (GenAI / LLM):** L'integrazione di un Large Language Model (es. GPT-4 o Llama 3) potrebbe trasformare il sistema da analitico a prescrittivo. Invece di limitarsi a dire «probabilità bassa», il sistema potrebbe riscrivere attivamente il titolo o la descrizione («blurb») per renderli più persuasivi e semanticamente coerenti con le campagne di successo della stessa categoria.
3. **Predizione Dinamica (Time Series):** Il modello attuale è statico (pre-launch). Un'evoluzione naturale sarebbe l'estensione a un modello dinamico che aggiorna la predizione giorno per giorno durante la campagna (es. al Day 1, Day 7), incorporando i dati di momentum iniziale (velocità di raccolta fondi, numero di backer nelle prime 24 ore) per raffinare la stima finale.